

HTML SVG

A Practical, From-Zero Learning Guide

Part 8 — SVG Paths in Depth

The `<path>` element is the most powerful drawing primitive in SVG. With one path you can create icons, letters, logos, curves, complex illustrations and precise custom shapes.

Goal: Understand every major SVG path command, how coordinates work, how Bézier curves and arcs are constructed, and how to think about paths mathematically.

1. The path Element

```
<svg viewBox="0 0 300 200">
  <path d="M 20 100 L 280 100"
        stroke="black"
        stroke-width="4"
        fill="none"/>
</svg>
```

The **d** attribute contains a sequence of commands and coordinates. The browser interprets that sequence as instructions for constructing the path.

2. The Path Command Alphabet

Command	Meaning
M / m	Move to
L / l	Line to
H / h	Horizontal line
V / v	Vertical line
Z / z	Close path
C / c	Cubic Bézier curve
S / s	Smooth cubic Bézier curve
Q / q	Quadratic Bézier curve
T / t	Smooth quadratic Bézier curve
A / a	Elliptical arc

3. Absolute vs Relative Commands

Uppercase commands use **absolute** coordinates. Lowercase commands use **relative** coordinates from the current point.

```
<!-- Absolute -->
M 50 50 L 150 50

<!-- Relative -->
m 50 50 l 100 0
```

Both examples describe the same horizontal movement when started from the origin.

4. The Current Point

A path always has a current point. Commands such as L, C and A start from that point and move it to a new location.

```
M 20 20
L 100 20
L 100 80
L 20 80
Z
```

The current point moves after each drawing command. **Z** closes the path back to the subpath's starting point.

5. M — Move To

```
<path d="M 30 40"/>
```

M moves the current point without drawing. It is commonly used to begin a subpath.

```
M 20 20
L 80 20

M 120 20
L 180 20
```

The second M starts another independent subpath.

6. Multiple Coordinates after M

```
M 20 20
50 20
80 20
```

After the first coordinate pair, additional coordinate pairs are treated as line-to commands. In practice, writing explicit L commands can make complex paths easier to read.

7. L — Line To

```
M 30 30
L 170 30
L 170 120
L 30 120
Z
```

L draws a straight line from the current point to the specified point.

8. H — Horizontal Line

```
M 30 50
H 270
```

H changes only the x coordinate while keeping y unchanged.

9. V — Vertical Line

```
M 150 20
V 180
```

V changes only the y coordinate while keeping x unchanged.

10. Z — Close Path

```
M 40 40
L 160 40
L 100 140
Z
```

Z closes the current subpath by connecting the current point to its starting point. It also creates a closed shape for filling.

11. Fill vs Stroke on Paths

```
<path d="M40 40 L160 40 L100 140 Z"
      fill="gold"
      stroke="black"
      stroke-width="5"/>
```

A closed path can have both an interior fill and an outline.

12. Multiple Subpaths

```
<path d="
  M 30 30 H 100 V 100 H 30 Z
  M 130 30 H 200 V 100 H 130 Z
"
fill="none"
stroke="black"
stroke-width="4"/>
```

One path element can contain multiple independent subpaths.

13. Nonzero and Evenodd Fill Rules

When paths overlap themselves or contain holes, the **fill-rule** controls which regions are filled.

```
<path d="..."
fill="black"
fill-rule="evenodd"/>
```

nonzero is the default. **evenodd** is often convenient for creating holes based on alternating inside/outside regions.

14. C — Cubic Bézier Curves

```
<path d="M 30 150
  C 80 20, 220 20, 270 150"
fill="none"
stroke="black"
stroke-width="4"/>
```

A cubic Bézier command uses two control points and an endpoint:

```
C x1 y1, x2 y2, x y
```

- Current point → start of curve.
- Control point 1 influences the beginning.
- Control point 2 influences the end.
- The final coordinate is the curve endpoint.

15. Understanding Cubic Control Points

Control points do not necessarily lie on the curve. They pull the curve toward them and control its direction and curvature.

```
M 40 140
C 80 20, 220 20, 260 140
```

The first control point controls the tangent leaving the start. The second controls the tangent approaching the end.

16. Cubic Curve with Relative c

```
M 40 140
c 40 -120, 180 -120, 220 0
```

The lowercase c interprets all coordinates relative to the current point.

17. S — Smooth Cubic Curve

```
M 30 120
C 70 20, 130 20, 170 120
S 270 220, 310 120
```

S creates a smooth cubic continuation. Its first control point is reflected from the previous cubic curve's second control point when applicable.

18. Why S is Useful

Without S, you must explicitly provide both control points for every cubic curve. S lets you continue a smooth curve with less data.

```
M 30 120
C 70 20, 130 20, 170 120
S 270 220, 310 120
```

19. Q — Quadratic Bézier

```
<path d="M 30 150
        Q 150 20, 270 150"
      fill="none"
      stroke="black"
      stroke-width="4"/>
```

A quadratic Bézier uses one control point and one endpoint:

```
Q x1 y1, x y
```

20. T — Smooth Quadratic Curve

```
M 30 120
Q 100 20, 170 120
T 310 120
```

T creates a smooth continuation by reflecting the previous quadratic control point.

21. Cubic vs Quadratic

Curve	Control points	Typical use
Quadratic	1	Simple smooth curves
Cubic	2	More precise complex curves

22. A — Elliptical Arc

Arcs draw portions of an ellipse or circle. The syntax is:

```
A rx ry x-axis-rotation large-arc-flag sweep-flag x y
```

The parameters look intimidating at first, but they become manageable once you understand the five important pieces.

23. Arc Radii

```
M 50 100
A 80 50 0 0 1 250 100
```

rx is the horizontal radius and **ry** is the vertical radius.

24. x-axis-rotation

```
M 50 100
A 80 40 30 0 1 250 100
```

This rotates the ellipse used for the arc by the specified angle.

25. large-arc-flag

This flag chooses between the smaller and larger possible arc connecting the endpoints.

```
A 80 80 0 0 1 220 100
A 80 80 0 1 1 220 100
```

The fourth parameter is either **0** or **1**.

26. sweep-flag

The sweep flag chooses the direction in which the arc is drawn.

```
A 80 80 0 0 1 220 100
A 80 80 0 0 0 220 100
```

The fifth parameter is either **0** or **1**.

27. The Four Arc Possibilities

For the same start point, endpoint and radii, changing the large-arc and sweep flags produces up to four different arc choices.

```
A 80 80 0 0 0 220 100
A 80 80 0 0 1 220 100
A 80 80 0 1 0 220 100
A 80 80 0 1 1 220 100
```

This is one of the most important things to experiment with when learning SVG arcs.

28. Drawing a Circle with Arcs

```
<path d="
  M 150 50
  A 100 100 0 1 1 149.99 50
"
fill="none"
stroke="black"
stroke-width="4"/>
```

A full circle can be constructed from two arcs. A tiny endpoint difference is often used to avoid ambiguous identical start and end points.

29. Drawing a Semicircle

```
<path d="
  M 50 100
  A 100 100 0 0 1 250 100
"
fill="none"
stroke="black"
stroke-width="4"/>
```

30. Combining Lines and Curves

```
<path d="
  M 40 160
  L 40 80
  Q 40 30 90 30
  H 210
  Q 260 30 260 80
  V 160
  Z
"
fill="none"
```

```
stroke="black"  
stroke-width="4"/>
```

Complex illustrations are normally built by combining several simple commands.

31. Relative Coordinates

```
M 50 50  
l 100 0  
l 0 80  
l -100 0  
Z
```

Relative coordinates are convenient when shapes need to be moved or generated programmatically.

32. When Absolute Coordinates Are Easier

- Tracing an illustration with known design coordinates.
- Editing a path manually in an SVG editor.
- Understanding exact locations in a fixed coordinate system.
- Debugging a path where each point needs to be obvious.

33. When Relative Coordinates Are Easier

- Generating repeated shapes with code.
- Creating reusable patterns.
- Writing compact paths.
- Moving a complete drawing while preserving its internal geometry.

34. Path Command Separation

```
M10,10 L100,10 L100,100 Z
```

Commas and spaces can separate numbers, and command letters separate command sequences. Keep formatting readable when learning; compact path data is easier to generate than to manually understand.

35. Path Length

SVG provides the **pathLength** attribute, which can normalize the conceptual length of a path.

```
<path d="M20 100 C100 20 200 20 280 100"  
      pathLength="100"  
      stroke="black"  
      fill="none"/>
```

This can be useful for animations and calculations that need a normalized scale.

36. getTotalLength()

In JavaScript, SVG paths expose methods for measuring their geometry.

```
const path = document.querySelector("path");  
  
const length = path.getTotalLength();  
  
console.log(length);
```

The returned length is useful for animations, drawing progress indicators and geometry calculations.

37. getPointAtLength()

```
const path = document.querySelector("path");

const point = path.getPointAtLength(100);

console.log(point.x, point.y);
```

This returns the point along the path at a specified distance.

38. Drawing Animation with stroke-dasharray

```
<path id="line"
  d="M20 100 C100 20 200 20 280 100"
  fill="none"
  stroke="black"
  stroke-width="5"
  stroke-dasharray="1000"
  stroke-dashoffset="1000"/>
```

By animating stroke-dashoffset toward zero, you can create a line-drawing effect. The exact dash values should be chosen relative to the path's length.

39. JavaScript Path Animation Idea

```
const path = document.querySelector("#line");
const length = path.getTotalLength();

path.style.strokeDasharray = length;
path.style.strokeDashoffset = length;

requestAnimationFrame(() => {
  path.style.transition = "stroke-dashoffset 2s";
  path.style.strokeDashoffset = "0";
});
```

40. Path Data in React

```
function Arrow() {
  return (
    <svg viewBox="0 0 200 100">
      <path
        d="M20 50 H150"
        fill="none"
        stroke="currentColor"
        strokeWidth="4"
      />
    </svg>
  );
}
```

In JSX, SVG attribute names such as **strokeWidth** use React's camelCase convention.

41. Building an Icon from Paths

```
<svg viewBox="0 0 24 24"
  width="48"
  height="48"
  fill="none"
  stroke="currentColor"
  stroke-width="2">

  <path d="M4 12
    L10 18
    L20 6"/>
```

</svg>

Many familiar icons are simply carefully designed path data.

42. Path Data and Design Tools

Design applications can export SVG paths automatically. You do not need to calculate every coordinate manually to become good at SVG. The important skill is understanding what the exported path commands mean and knowing how to edit or generate them.

43. Path Simplification

A design tool may export a path containing many points. Simplifying unnecessary points can reduce file size and make paths easier to maintain, but excessive simplification can change the visual result.

44. Common Path Mistakes

- Forgetting that lowercase coordinates are relative.
- Using the wrong current point.
- Mixing up control points and endpoints.
- Choosing the wrong arc flags.
- Expecting Z to draw a straight line to an arbitrary point.
- Creating an open path when a filled closed shape was intended.
- Using too many points when a Bézier curve would be cleaner.
- Trying to understand a long minified path before formatting it.

45. A Good Path-Learning Workflow

- Start with M, L, H, V and Z.
- Then learn Q and T.
- Then learn C and S.
- Finally master A.
- Practice both absolute and relative commands.
- Draw each command separately before combining them.
- Use a browser and developer tools to inspect your results.
- Use design software as a visual reference, but understand the generated d attribute.

46. Mini Project — Heart

```
<svg viewBox="0 0 200 180">
  <path d="
    M100 160
    L30 90
    C-5 55 20 15 60 25
    C82 30 95 48 100 60
    C105 48 118 30 140 25
    C180 15 205 55 170 90
    Z"
  />
```

```
        fill="tomato"/>
</svg>
```

Experiment with the control points to change the shape of the heart.

47. Mini Project — Cloud

```
<svg viewBox="0 0 300 180">
  <path d="
    M60 130
    C25 130 25 80 65 80
    C70 35 135 25 155 70
    C190 45 240 70 235 110
    C270 115 265 150 230 150
    H60
    Z"
    fill="skyblue"/>
</svg>
```

48. Mini Project — Custom Arrow

```
<svg viewBox="0 0 300 120">
  <path d="
    M30 60
    H210
    M170 25
    L210 60
    L170 95"
    fill="none"
    stroke="black"
    stroke-width="8"
    stroke-linecap="round"
    stroke-linejoin="round"/>
</svg>
```

49. Mini Project — Curved Connector

```
<svg viewBox="0 0 500 220">
  <path d="
    M80 170
    C180 170 150 40 250 40
    S320 170 420 170"
    fill="none"
    stroke="royalblue"
    stroke-width="6"/>
</svg>
```

This kind of connector is useful in dashboards, flowcharts and node-based interfaces.

50. Exercises

- Draw a triangle using M, L and Z.
- Draw a rectangle using only M, H, V and Z.
- Create a wave using C.
- Create a smooth S-shaped path using C and S.
- Create a quadratic wave using Q and T.
- Experiment with all four arc flag combinations.
- Build a circle from two arcs.
- Create a custom map pin using paths.

- Create a simple shopping-cart icon using paths.
- Animate a path using `getTotalLength()` and `stroke-dashoffset`.
- Build your own logo using only path commands.

51. Quick Reference

Command	Syntax idea	Purpose
M	M x y	Move
L	L x y	Line
H	H x	Horizontal line
V	V y	Vertical line
Z	Z	Close subpath
C	C x1 y1 x2 y2 x y	Cubic Bézier
S	S x2 y2 x y	Smooth cubic
Q	Q x1 y1 x y	Quadratic Bézier
T	T x y	Smooth quadratic
A	A rx ry rot large sweep x y	Elliptical arc

52. What's Next?

Part 9 will cover **SVG Animation & Interaction**: CSS animations, transitions, SMIL animation, JavaScript-driven SVG, motion along paths, hover effects, drawing animations, interactive graphics and practical animated UI projects.